Universität Siegen

Naturwissenschaftlich-Technische Fakultät

Master's Thesis

for acquiring the academic degree of Master of Science Computer Science

Evaluating and Improving Software Development Processes in Large-Scale Projects Using LLM-Based AI Agents: A Case Study of CrypTool

Betreuer	Prof. Bernhard Esslinger
Erstprüfer	Prof. Bernhard Esslinger
Zweitprüfer	Prof. Dr. Roland Wismüller
vorgelegt von	Michael Heep
Matrikelnummer	1272000
Abgabedatum	tbd

Kurzzusammenfassung

Abstract

Contents

Kurzzusammenfassung 2

Abstract 2

Contents 3

1 Introduction 4
- 1.1 Motivation and Problem Statement . . . 4
- 1.2 Research Questions . . . 5
- 1.3 Research Approach . . . 6
- 1.4 Limitations for this Master's Thesis . . . 8
- 1.5 Structure . . . 8

2 Background and Related work 10

3 Foundations 11
- 3.1 Large Language Models (LLMs) . . . 11
- 3.2 Agentic AI . . . 11
- 3.3 Skills and Tooling . . . 12
- 3.4 Requirements in the CTO developement project . . . 12

4 Developing an Agent Setup for CrypTool-Online 13

5 Implementation and Case Studies 14
- 5.1 Experimental Setup . . . 14
- 5.2 Case Study: Claude Code . . . 14
- 5.3 Case Study: Codex . . . 14
- 5.4 Case Study: OpenCode . . . 14

6 Evaluation 15

7 Conclusion and Future Work 16

Bibliography 17

List of Abbreviations 18

Eidesstattliche Erklärung 19

Content of the CD 19

1 Introduction

1.1 Motivation and Problem Statement

Within a few years, coding assistants based on large language models (LLMs) have evolved from line-by-line autocompletion into increasingly autonomous software-engineering agents. Tools such as Claude Code, Codex, or OpenCode can inspect existing repositories, plan and implement changes across multiple files, execute build and test procedures, and react to the results within a single task given in natural language. Both companies and open-source projects are adopting these agents in the hope of accelerating software development. However, empirical evidence regarding their effectiveness in practice is still limited. Most research so far is benchmark-driven: Benchmarks such as SWE-bench [2] measure whether an agent can resolve individual GitHub issues, but they provide little insight about sustained contribution to a real project with its own conventions, tooling, and development history. Moreover, there is little guidance on how such agents should be configured and integrated into an existing project. Initial exploratory studies on the configuration of agentic coding tools [1, 4, 3] indicate that practitioners are developing best practices in an ad hoc manner, without an established body of knowledge to draw on.

A second, older problem exists independently of so called artificial intelligence (AI) tooling. **[TODO: Introduce AI tooling before using the term.]** A developer who joins a large software project spends a considerable part of their initial effort not on writing code but on learning: the structure of the repository, the conventions of the code base, the build and test tooling, and the existing abstractions that a new feature is expected to reuse. This ramp-up cost grows with the age and size of the project, because mature projects accumulate conventions that are rarely written down completely. CrypTool-Online (CTO) is such a project. It is a large, long-lived web application that offers a collection of cryptographic apps for educational purposes. The code base comprises approximately 79,000 lines of code[1]. The CrypTool project itself started in 1998 with its offspring CTO being in development for over ten years. The CTO project has an established but only partially documented pattern for how new apps are structured and added. Open-source projects like CTO depend particularly on lowering this barrier, since their contributors are often volunteers and students who can only invest limited time before their first contribution has to succeed.

[1] Physical lines excluding blank lines and comments, counted with `cloc` 2.10 on commit `9d297a17` (2026-10-20) of the cryptool.org repository; tests (approximately. 8,700 lines) included; third-party UI snippets and data files excluded. The 312 MDX content pages, which hold approximately 10,600 non-blank lines, are not included in this figure.

LLM-based agents appear to be a natural fit for this onboarding problem. They can explore a repository within minutes, infer conventions from existing code, and produce a first implementation that follows those conventions. Yet it is unknown how well they actually perform this in a project of the size of CTO, which kind of setup makes a difference (a plain prompt, project-specific instruction files, additional skills or tools), and where they still fail or require human intervention. This thesis investigates these questions by letting several agents implement new CTO apps under different configurations and by comparing the resulting processes and outcomes in a qualitative multi-case study. The research questions guiding this investigation are stated in the following subsection.

1.2 Research Questions

The overarching goal of this thesis is to understand whether, and under which conditions, LLM-based agents make it easier for a new contributor to work productively in a large existing software project. This goal is broken down into the following research questions:

RQ1 Can LLM-based agents make it easier for a new contributor to work in a large software project such as CrypTool-Online?

RQ2 What is possible with current cloud-based LLM agents in this context, and what are their limitations?

RQ3 What is possible with current offline LLM agents in this context, and what are their limitations?

RQ4 What is the effect on a reviewer once such an agent-supported process is applied?

RQ1 is the core question of this thesis and is answered through the case studies. RQ2 and RQ3 refine it with regard to the available technology: They motivate the selection of agents and define what has to be observed during the experiments, namely where an agent succeeds autonomously, where it gets stuck, and where it needs intervention. RQ4 addresses a consequence that is easily overlooked: When an agent writes the code, the human effort shifts from writing to reviewing. Whether this shift is a net benefit for the project can only be judged if the reviewer's perspective is evaluated as well.

1.3 Research Approach

The research questions are addressed with an exploratory, qualitative multi-case study. This design is chosen for three reasons. First, the use of LLM-based agents in large software projects is a young field without established metrics for what it means to "contribute more easily". The study is therefore exploratory, which means that rather than testing predefined hypotheses, it aims to identify which effects occur and which criteria matter [5]. Second, the behavior of an agent depends strongly on the individual project, the concrete task, and the interaction with the developer. This is difficult to capture in aggregated numbers but well suited to detailed qualitative case descriptions. Third, a controlled experiment with enough runs for statistically meaningful results is beyond the scope of a single thesis, since each run involves a real CTO app and a manual review of the result. Instead, a small number of cases is studied in depth and compared with each other.

Besides the case studies, the thesis has a constructive part. Based on an analysis of the CTO code base and its conventions, a project-specific agent setup is developed (Section 4). Its core is a lean instruction file (`AGENTS.md`) that contains only what an agent cannot infer from the code itself. Recurring procedures are moved into skills that the agent loads on demand, namely the skills provided by Chakra UI, the component library used by CTO, and project-specific skills for creating, describing, and checking a new app. In addition, the setup connects the agent to tools for looking up the Chakra UI documentation and for testing an app in the browser. This division follows the observation that long context files tend to lower the success rate of agents and raise their costs [**Gloaguen2026TODO**]**[TODO: Add Gloaguen et al. to references.bib and fix the citation key.]**. The same setup is provided to all three agents. It is finalized before the case studies and is not changed between runs, so that the cases remain comparable. The thesis therefore produces two kinds of results: the empirical findings of the case studies, which answer the research questions, and the agent setup itself, together with guidelines on how such a setup can be built for other large software projects.

The unit of analysis is a single case, defined as one agent implementing one CTO app under one configuration. Two configurations are compared: a baseline, in which the agent works on the repository with its existing generic instruction files removed and thus without any context file, skills, or additional tools, and the configured setup, in which the agent additionally has access to the agent setup described above. For the first app, both configurations are run with three agents: Claude Code and Codex, which use cloud-based models, and OpenCode, which uses a locally hosted model and thereby addresses RQ3. **[TODO: Name the local model and the hardware it runs on.]** Since

these six cases implement the same app, differences between them can be attributed to the agent and the configuration rather than to the task. The second app is implemented only with Claude Code, again in both configurations. It is chosen to differ from the first app in its complexity and serves as a replication: It shows whether the effect of the agent setup observed for the first app also occurs for a different task. **[TODO: Name both apps and justify their selection.]** This results in eight cases, with Claude Code as the primary case that is studied on both apps and described in the most detail. The experimental setup is described in Section 5.

Each case produces several kinds of data. All runs for the same app start with the same initial prompt, which describes the app to be implemented, and is continued with follow-up prompts until the result is accepted or the run is aborted. **[TODO: Add the initial prompt to the appendix and define the acceptance and abort criteria.]** During a run, the session transcript is recorded and observations are noted, in particular the number of prompts and iterations, the elapsed time, the token usage, the points at which the agent got stuck, and the interventions by the developer. After a run, the resulting code is kept as an artifact, together with the results of the project's linter and tests. Finally, the result is reviewed by a developer who is familiar with the CTO code base, and the reviewer's assessment is recorded. The review is blind, which means that the results are rated without knowing which configuration produced them. **[TODO: Describe who reviews (author, CTO maintainer, both?) and in which form the assessment is recorded.]**

The analysis proceeds in two steps. In the first step, each case is evaluated against a common set of criteria, such as the autonomy of the agent, the adherence to the conventions of the CTO code base, the reuse of existing code, and the correctness and maintainability of the result. These criteria are defined in Section 6. In the second step, the cases are compared with each other in three directions: across the three agents within the same configuration, across the two configurations of the same agent, and, for Claude Code, across the two apps. The comparison of configurations shows whether the agent setup has an effect at all and whether this effect is similar for all agents. The comparison of apps shows whether this effect depends on the task. **[TODO: Decide whether the "developing without an agent" baseline (see note in evaluation section) is part of the comparison.]**

With regard to the research questions, RQ1 is answered by the case studies as a whole, in particular by the comparison of the two configurations and its replication on the second app, RQ2 by the observations of where Claude Code and Codex succeed and where they fail, RQ3 by the same observations for OpenCode with the locally hosted model, and RQ4 by the reviewer's assessment of the results.

Agent	Version
Claude Code	2.1.x
Codex	2.0
OpenCode	3.0

Table 1: Versions of the agents and models used in this thesis.

1.4 Limitations for this Master's Thesis

The scope of this thesis is deliberately limited in several respects. Not all available agents can be covered. Instead, three agents are selected: Claude Code and Codex as representatives of cloud-based agents, and OpenCode as an open-source agent that can be run with a locally hosted model. Likewise, only two configurations are compared, the unconfigured agent and the agent with the developed setup. Since the setup is evaluated as a whole, the individual effects of the instruction file, the skills, and the tools cannot be separated. In addition, only two CTO apps are implemented, and the second one only with Claude Code. The comparison between agents therefore rests on a single app, and the results may differ for apps of other complexity. Each case is run only once. Since agents behave non-deterministically, a repeated run with the same agent and configuration may lead to a different result. Because OpenCode is the only agent that uses a local model, differences between OpenCode and the other agents cannot be attributed to either the agent or the model alone. Furthermore, the author both develops the agent setup and evaluates the cases, which may bias the assessment. The blind review reduces this bias but cannot remove it entirely, since the author also defined the evaluation criteria. The results are therefore illustrative rather than statistically generalizable. Moreover, both the agents and the underlying models evolve rapidly. Consequently, the observations reflect the versions available at the time of writing and may not hold for future releases.

1.5 Structure

The remainder of this thesis is organized as follows. Section 2 reviews related work on LLM-based agents in software engineering and on the configuration of agentic coding tools. Section 3 introduces the necessary foundations: large language models, agentic AI, skills and tooling, and the requirements of the CTO development project. Section 4 describes the development of the customized agent setup that is used in the case studies. Section 5 presents the experimental setup and the case studies for each agent. Section 6

evaluates and compares the results across agents and approaches. Section 7 concludes the thesis and outlines future work.

2 Background and Related work

3 Foundations

3.1 Large Language Models (LLMs)

Short intro to what LLMs are, how they work, and where they came from. Mention SOTA LLMs and their capabilities.

3.1.1 Online and Offline LLMs

Compare and contrast online and offline LLMs, their use cases, and limitations. Online is fast, powerful and often costs money Offline is slower, less powerful, but more private and secure and costs no money (besides the hardware and electricity) Explain how offline LLMs can be used in a local environment (e.g. using LM Studio), mention what hardware is needed to run them, and what the limitations are (e.g. no internet access, limited context window, etc.)

3.2 Agentic AI

A short description of what an agent is. Short outline of the history/evolution of agents. In contrast to raw models (LLMs), an agent is a complete functional system that combines a model with an outer operational stucture

3.2.1 Harnessing LLMs

Short description: Harness is the software infrastructure and runtime layer that wraps around the raw model (LLM), giving it access to system prompts, file systems, code execution and other tools important: autonomous execution loop

3.2.2 Complete Agents

LLM and Harness together form a complete agent.

3.3 Skills and Tooling

3.4 Requirements in the CTO developement project

4 Developing an Agent Setup for CrypTool-Online

5 Implementation and Case Studies

5.1 Experimental Setup

5.2 Case Study: Claude Code

5.3 Case Study: Codex

5.4 Case Study: OpenCode

6 Evaluation

7 Conclusion and Future Work

Bibliography

[1] Matthias Galster, Seyedmoein Mohsenimofidi, Jai Lal Lulla, Muhammad Auwal Abubakar, Christoph Treude, and Sebastian Baltes. "Configuring Agentic AI Coding Tools: An Exploratory Study". In: *Proceedings of the 3rd ACM International Conference on AI-Powered Software* (2026). URL: https://api.semanticscholar.org/CorpusID:289930025 (cit. on p. 4).

[2] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" In: *ArXiv* abs/2310.06770 (2023). URL: https://api.semanticscholar.org/CorpusID:263829697 (cit. on p. 4).

[3] Jai Lal Lulla, Matthias Galster, Jie M. Zhang, Sebastian Baltes, and Christoph Treude. "The Impact of Configuring Agentic AI Coding Tools on Build-vs-Buy Decisions: A Study Protocol". In: *ArXiv* abs/2606.03907 (2026). URL: https://api.semanticscholar.org/CorpusID:288903941 (cit. on p. 4).

[4] Jai Lal Lulla, Seyedmoein Mohsenimofidi, Matthias Galster, Jie M. Zhang, Sebastian Baltes, and Christoph Treude. "On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents". In: *ArXiv* abs/2601.20404 (2026). URL: https://api.semanticscholar.org/CorpusID:285101666 (cit. on p. 4).

[5] Per Runeson and Martin Höst. "Guidelines for conducting and reporting case study research in software engineering". In: *Empirical Software Engineering* 14 (2009), pp. 131–164. URL: https://api.semanticscholar.org/CorpusID:207144526 (cit. on p. 6).

List of Abbreviations

AI artificial intelligence
CTO CrypTool-Online
LLM large language model

Eidesstattliche Erklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe, insbesondere keine anderen als die angegebenen Informationen aus dem Internet. Diejenigen Paragraphen der für mich geltenden Prüfungsordnungen, die etwaige Betrugsversuche betreffen, habe ich zur Kenntnis genommen.

Der Speicherung meiner Masterarbeit zum Zweck der Plagiatsprüfung stimme ich zu. Ich versichere, dass die elektronische Version mit der gedruckten Version inhaltlich übereinstimmt.

(Datum) (Unterschrift)

Content of the CD

- Master's thesis as PDF
- Source code of the application